TD1: Neural networks in depth

(In class exercise, evaluated)

Together : Classify data with a multi-layer perceptron

Objective: Observe the inner functioning of a perceptron in a classification task.

Open the website http://playground.tensorflow.org/

Select the third type of data (on the left side of the website). It contains 2 groups of points (orange and blue).

Train the network with no hidden layer. Can it classify the observations?

Do the same with the second (top right) and then the first dataset (top left). What do you observe? Modify the network architecture in order improve the classification. Observe the "sub-classifications" made by the hidden layers of the network. **Warning: for each new learning, don't forget to réinitialize the network!**

Find an architecture that allows to classify the last dataset. In this case, the training is not instant! Leave it enough time to be optimized.

Exercise 1 : Classify data with a multi-layer perceptron

Objective: Observe the inner functioning of a perceptron in a classification task.

1. Open the website http://playground.tensorflow.org/
2. Select the third type of data (on the left side of the website). It contains 2 groups of points (orange and blue).
3. Train the network with no hidden layer. Can it classify the observations?
4. Do the same with the second (top right) and then the first dataset (top left). What do you observe? Modify the network architecture in order improve the classification. Observe the "sub-classifications" made by the hidden layers of the network. **Warning: for each new learning, don't forget to re-initialize the network!**
5. Find an architecture that allows to classify the last dataset. In this case, the training is not instant! Leave it enough time to be optimized.

Exercise 2: Simply with sci-kit learn

Objective: load, visualize a dataset. Train a logistic classifier and a multilayer perceptron with python and scikit learn

1. In Python, load the iris dataset. Use a pairplot polt (seaborn library) to visualize and analyze the data.
2. Separate the data into 4 matrices: x_train, x_test, y_train, y_test. Input data are on the first 4 columns and the output data is on the last column. Train/test ratio must be 75/25.
3. By using SKLearn, build a logistic classifier on the training data set. Evaluate the trained model on the test dataset and display the prediction score. What do you think about this score?
4. Transform the data to make them usable by a neural network: the output must be a Nxn matrice (n being the number of classes of the problem and N the number of observations) instead of a Nx1 vector. Each column must contain a 1 if the observation is of this class and zero otherwise. Look at the *to_categorical* function!
5. Still using SKLearn, create a perceptron to classify the IRIS dataset. The network must contain a hidden layer of 16 neurons. You can change the hyper-parameters (solver, activation function, learning rate, etc.) of the network as you wish.
6. Train the network on the training dataset and display the score on both the training and the test datasets.

Exercise 3: With more control using keras

Objective: Getting started with Keras and build a simple neural network.

1. Strat from (or redo) categorized data from the IRIS data set.
2. Using Keras, create a perceptron built as follows:
   a. One dense layer of 16 neurons connected to a 4-neurons input layer. The activation function of this layer must be a sigmoid
   b. A output layer of 3 neurons with a softmax activation function.
   c. Used a categorical crossentropy (log loss) objective function, the adam optimizer and precompute the accuracy metric.
3. Train the network for 100 epochs
4. Compare the precision of the classification obtained with the logistic regression of exercise 1.

Exercise 4 : Adapt a network to a given problem

Objective: Build a multilayer perceptron by changing its architecture to be adapted to a given problem

1. Load the MNist dataset and visualize the different loaded vectors. Display the first images of the dataset. What is this dataset about?
2. Modify the data to:
   a. Linearize the data in two dimensions.
   b. Normalize the data
   c. Transform the class vector to be used by a neural network.
3. Train the architecture of the previous exercise to this new problem. Make sure to adapt the input and output layer sizes.
4. Train the network of few epochs with a batch size of 128. Observe the network score. What do you think about it?

Homework

Next class – show results in a 10 minutes presentation

Will be evaluated on:

1. Show the code and explain it.
2. Comment on the results.
3. What were the main challenges you have faced?

Exercise 5: Deeper multi-layer perceptron

Objective: Create a multi-layer perceptron on a more complex image classification task.

1. Load, display and format the Cifar10 dataset
2. Build the mono-layer perceptron from the previous exercise. Train, evaluate and display classification errors. Try to modify gently the network (no new layer but larger hidden layer, different activation functions, etc.) and evaluate these different networks. What do you think of the prediction you obtain?
3. Build the following multilayer perceptron:
   a. First hidden layer of 512 neurons densely connected to the input layer, *relu* activation function, dropout of 0.2 and batch normalization.
   b. Second hidden layer of 512 neurons densely with *relu* activation function, batch normalization and dropout of 0.2.

4. Run 50 epochs of training and evaluate the obtained network. What do you think about the prediction score?
5. Save your model on your hard drive and build the accuracy and loss for each training epoch.
6. Build the architecture! Comment on advantages of your choice and the disadvantages.